

Unidade 2 — Arquitetura, REST, GraphQL e Integrações

Resumo otimizado para estudo, provas e entrevistas

1. Estilos Arquiteturais de APIs

As APIs podem seguir dois modelos principais de comunicação:

Request/Response: o cliente solicita e o servidor responde. Exemplos: REST, SOAP, GraphQL e gRPC.

Event Driven: comunicação baseada em eventos. Exemplos: WebSockets e Webhooks.

Comparativo dos Principais Estilos

SOAP: protocolo baseado em XML, robusto, seguro e comum em ambientes corporativos.

REST: estilo arquitetural mais popular, baseado em recursos e HTTP.

GraphQL: cliente escolhe exatamente quais dados deseja receber.

gRPC: alta performance usando HTTP/2 e Protocol Buffers.

WebSockets: comunicação bidirecional em tempo real.

Webhooks: notificações baseadas em eventos.

2. REST x GraphQL

REST

- Múltiplos endpoints
- Cache simples
- Versionamento nativo
- Usa status HTTP naturalmente
- Suporta JSON, XML e outros formatos

GraphQL

- Endpoint único
- Cliente define exatamente os dados desejados
- Evita overfetching e underfetching
- Trabalha principalmente com JSON
- Mais flexível para aplicações complexas

3. Representação de Dados

Formatos mais utilizados:

- JSON — padrão dominante em APIs modernas.
- XML — comum em SOAP e sistemas legados.
- YAML — muito usado em configuração e documentação.
- Protocol Buffers — formato binário utilizado pelo gRPC.

4. Documentação de APIs

Uma boa documentação deve explicar endpoints, parâmetros, respostas, exemplos e códigos HTTP.

Principais padrões:

- OpenAPI (Swagger)
- API Blueprint
- RAML

- WADL
- WSDL (SOAP)

5. JSON e JSON Schema

JSON é o formato mais utilizado para troca de dados entre APIs. Baseia-se em pares chave-valor e suporta:

- String
- Number
- Boolean
- Objects
- Arrays

JSON Schema

Utilizado para documentar e validar documentos JSON.

Benefícios:

- Garantia de qualidade dos dados
- Automação de testes
- Documentação padronizada

Palavras-chave importantes:

- \$schema
- \$id
- \$ref
- \$defs

6. SOAP

SOAP é um protocolo baseado em XML. Muito utilizado em governos, bancos e sistemas corporativos.

Estrutura:

- Envelope (obrigatório)
- Header (opcional)
- Body (obrigatório)

Componentes WSDL:

- Types
- Messages
- PortType
- Binding
- Service

7. REST

REST é um estilo arquitetural baseado em recursos. APIs REST completas são chamadas RESTful.

Princípios REST

- Cliente e servidor independentes
- Stateless
- Cache
- Interface uniforme
- Sistema em camadas

CRUD REST

GET /usuarios

POST /usuarios

GET /usuarios/{id}

PUT /usuarios/{id}

PATCH /usuarios/{id}
DELETE /usuarios/{id}

Status HTTP mais cobrados

200 OK
201 Created
204 No Content
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
500 Internal Server Error

8. Conceitos REST Avançados

Stateless

O servidor não guarda sessão. Cada requisição deve conter todas as informações necessárias.

HATEOAS

A resposta contém links que permitem navegar pela API.

Versionamento

Exemplo: /v1/cursos e /v2/cursos

Cache

Headers importantes: Cache-Control, ETag, Last-Modified, Expires e Age.

9. Modelo de Maturidade REST

Nível 0 → Endpoint único

Nível 1 → Recursos separados

Nível 2 → Recursos + métodos HTTP corretos

Nível 3 → Recursos + HTTP + HATEOAS

10. GraphQL

Principais vantagens:

- Endpoint único
- Menos requisições
- Menos tráfego de rede
- Cliente controla os dados recebidos
- Elimina overfetching e underfetching

Estrutura:

Schema → Queries/Mutations → Resolvers → Banco/APIs

11. WebSockets

Comunicação bidirecional e persistente. Após o handshake HTTP, a conexão permanece aberta.

Casos de uso:

- Chats
- Jogos online
- Dashboards em tempo real
- Notificações instantâneas

12. Webhooks

Integração orientada a eventos. Em vez de polling constante, o servidor notifica quando algo acontece.

Casos comuns:

- Pagamentos aprovados
- Eventos do GitHub
- Automações entre sistemas

Resumo para Entrevistas

- REST continua sendo o padrão do mercado.
- GraphQL resolve overfetching e underfetching.
- gRPC é excelente para microsserviços.
- SOAP ainda existe em ambientes corporativos.
- WebSockets oferecem comunicação em tempo real.

- Webhooks eliminam polling desnecessário.
- JSON é o formato dominante das APIs modernas.